P2299R0
`mdspan` and CTAD

Published Proposal, 2021-02-27

Author:

Bryce Adelstein Lelbach (he/him/his) — Library Evolution Chair (NVIDIA)

Source:

GitHub

Issue Tracking:

GitHub

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Audience:

WG21

Table of Contents

§ 1. Introduction

[P0009R10]'s `mdspan` is a convenience template alias for simpler use cases of `basic_mdspan`:

```
template <class ElementType, ptrdiff_t... Extents>
using mdspan = basic_mdspan<ElementType, extents<Extents...>>;
```

In the `basic_mdspan`/`span` interface, extents can be either static, e.g. expressed at compile time:

```
mdspan<double, 64, 64> a(data);
```

or dynamic, e.g. expressed at run time:

```
mdspan<double, dynamic_extent, dynamic_extent> a(data, 64, 64);
```

You can also use a mix of the two styles:

```
mdspan<double, 64, dynamic_extent> a(data, 64);
```

§ 2. Problem

The [P0009R10] interface style for expressing extents currently interacts poorly with Class Template Argument Deduction (CTAD). As of C++20, CTAD now works with template aliases, so you can write this:

```
mdspan a(data, 64, 64);
```

This syntax would be very nice. It would remove the need to explicitly spell out the verbose `dynamic_extent` when dealing with run time extents, which I believe is the common case for most users.

However, the above code does not appear to do what you might expect. This appears to instantiate `mdspan<double>`, and then `basic_mdspan<double, extents<>>`; a multi-dimensional array of rank 0. This will lead to a static assertion as `basic_mdspan`'s dynamic extent constructor. You can see the code on Godbolt here.

§ 3. Solutions

If `mdspan` was a template class, not a template alias, we could simply add a deduction guide to handle this:

```
template <class ElementType, class... IndexType>
explicit mdspan(ElementType*, IndexType...)
  -> mdspan<ElementType, [] (auto) constexpr
                         { return dynamic_extent; }
                         (identity<IndexType>{})...>;
```

However, it seems you cannot add a deduction guide for a template alias. Perhaps there is a way of formulating a deduction guide for `basic_mdspan` that would help here, however I could not find a way of doing this.

One way we could solve this problem would be to make `mdspan` a template class, not a template alias, and then give that template class such a deduction guide. You can see a sketch of this on Godbolt here.

However, making `mdspan` a template class instead of a template alias would introduce all sorts of other challenges, as it would become a distinct type from `basic_mdspan` and functions taking a `basic_mdspan` would not take an `mdspan`. Perhaps we could come up with a solution, possibly involving conversions, that would be acceptable.

§ 4. P.S. No `ptrdiff_t`

A final, partially related note: [P0009R10] must be updated to use `size_t` instead of `ptrdiff_t`. As much as I love signed types, `dynamic_extent`, which shipped in `<span>` in C++20, is defined as a `size_t`. If `extents`, `mdspan`, and `basic_mdspan` use `ptrdiff_t`, users will experience all sorts of signedness and narrowing warnings when trying to use `dynamic_extent` with these facilities.

§ References

§ Informative References

[P0009R10]

Christian Trott, Bryce Adelstein Lelbach, Daniel Sunderland, D. S. Hollman, H. Carter Edwards, Mauro Bianco, Ben Sander, Athanasios Iliopoulos, John Michopoulos, Mark Hoemmen. mdspan. 28 February 2020. URL: https://wg21.link/p0009r10